Firmware review: Moon reading light (ATtiny3216 + TLC59116 + LM75B)

Independent reviewer, 2 Nov 2026. Scope: `/home/user/reading_light/firmware` against the owner's spec (PLAN.md R5–R9, R17–R20) and the datasheets in `scratchpad/ds/` (TLC59116 SLDS157E, LM75B Rev 6.1, ATtiny3216 DS40002205A). I did not edit any repo file.

Snapshot reviewed. `src/main.cpp` (16:44), `lib/core/*` (16:32), `include/modes_table.h` and `tools/gen_modes.py` (16:28), `include/board_map.h` (16:31). **During the review, another session regenerated `include/board_map.h` (17:28)** through a new `write_board_map()` in `hardware/build.py`. That closes FW-01 in the working tree; I re-verified it (see FW-01). No other firmware file changed during the review (checked with `diff -r` against my snapshot copy).

What I ran

Check	Result
`pio test -e native` (repo)	17/17 pass (before and after the 17:28 change)
`pio run -e attiny3216` / `-e attiny3216_debug`	Both build. Flash 9,542 B (29.1 %), RAM 256 B (12.5 %). Debug build: 11,393 B / 401 B
Compile `src/main.cpp` and `lightcore.cpp` **without** the framework's `-w`	1 real warning: `'t' may be used uninitialized` (main.cpp:177). `-Wconversion` adds only noise
Scratch suite `review/fwtest/run.sh` (10 tests: core logic + a TLC59116 register model driven by a verbatim copy of main.cpp's TLC code)	**Repo:** 8 fail. All 10 failed on the 16:31 snapshot; the 2 F01 tests pass since 17:28. **Patched copy:** 10/10 pass
`review/fwtest/run_fulltick` (1 ms ticks for 2^31 ms, no time jump)	Reproduces the 24.86-day 5 Hz strobe (FW-02)
simavr cycle counts (`review/fwtest/cycles/`)	`level_fraction` 6.3 k cycles; `mode_targets` 11.1 k; `core_tick` 2.6 k steady, **13.4 k while the thermal cap slews**; `tlc_output` maths 12.9 k
Proposed patch (`review/patches/firmware-review.patch`) applied to a scratch copy	AVR build clean (no project warnings with `-w` removed): 10,204 B / 263 B. Repo tests 17/17. Scratch tests 10/10. Applies cleanly (`patch -p1 --dry-run`) to the current repo tree

Findings summary

ID	Sev	Title	Status
FW-01	BLOCKER	`board_map.h` did not match the LED board: every mode lit the wrong LEDs, and the red night light lit warm-white + green	**Fixed in the working tree at 17:28 (not by me); verified**
FW-02	BLOCKER	`blink_until` wraps: after 24.86 days on, the lamp strobes full on/off at 5 Hz for 24.86 days. After 49.7 days, a re-fade from 0	open, patch
FW-03	MAJOR	Mode recipes were computed for the wrong RGB LED (Kingbright 5050, not the fitted Würth 3528) at 30 mA. Candle "1900 K" comes out yellow-green	open, patch
FW-04	MAJOR	`I_CHANNEL_MA` = 30.2 mA; REXT 750 Ω gives 25.0 mA	open, patch
FW-05	MAJOR	LM75B is left in shutdown after the battery-empty → USB resume, so thermal derating is frozen at a stale value	open, patch

ID	Sev	Title	Status
FW-06	MAJOR	No watchdog: a hung MCU leaves the LEDs at a stale level with no thermal or battery protection	open, patch
FW-07	MAJOR	TLC59116 errors are ignored: init result unchecked, `tlc_ok` never cleared, LED-board POR never detected (dark until power cycle), a failed IREF write leaves the gain 2× wrong	open, patch
FW-08	MAJOR	VDD measurement averages the first INTREF conversion, and resume trusts one unfiltered sample (false-resume / late-empty risk)	open, patch + bench
FW-09	MAJOR	BOD, EESAVE and SUT are never programmed: `pio run -t upload` writes no fuses, so the factory BOD stays **off**	open, process
FW-10	MAJOR (system)	Empty-battery "sleep" leaves the LED board powered. TLC59116 low-power ICC is unspecified; the sleep may drain the cell to PCM cut-off	open, measure / hardware
FW-11	MINOR	IREF is written before PWM, giving a 2× brightness flash (~1.8 ms) at each gain step up (R19)	open, patch
FW-12	MINOR	PIT is used with the RTC disabled: ATtiny3216 Rev A erratum means the USB wake may never fire	open, bench
FW-13	MINOR	`static last_slew` in `core_tick` is shared across instances and test runs	open, patch
FW-14	MINOR	CPU budget: `powf` and float maths in the 1 ms tick; 1.34 ms per tick while slewing	open
FW-15	MINOR	No I2C bus recovery at boot (required once a WDT exists)	open, patch
FW-16	MINOR	LM75B bytes are combined with unsequenced `Wire.read()` calls; no plausibility check	open, patch
FW-17	MINOR	Uninitialised `t` is passed to `core_temperature`	open, patch
FW-18	MINOR	Build hygiene: `-w` hides all warnings; host env has no warnings; `main.cpp` has no host coverage	open, patch
FW-19	MINOR	Soft-orange night mode uses the 2700 K phosphor white, which contains blue/green; R20 says "no green or blue light"	owner decision
N-1… N-12	NOTE	See the end	—

BLOCKER

FW-01: `include/board_map.h` did not match the LED board (fixed in the working tree during the review)

- **Where:** `firmware/include/board_map.h:5` (16:31 version).
- **Issue.** The firmware mapped `OUT_GROUP = {0,0,0,0, 1,1,1,1, 2,3,4, 2,3,4, -1,-1}`.
  - The LED board as designed is `{2,1,0,1, -1,-1, 0, 2,3,4, 1,0,1,0, 3,4}`. Three sources agree on this:
    1. U1 pad nets in `hardware/led-board/led-board.kicad_pcb`;
    2. the `kicad-cli sch export netlist` of the schematic (both the 16:59 and 17:28 versions);
    3. `design.out_group()`.

    - The header claimed to be generated by `hardware/build.py`, but at review time `build.py` contained no generator.
- **Evidence.** `fwtest/test_review.cpp` `test_F01_*`, run on the 16:31 header. Each line shows the firmware's drive per group, then the LEDs that actually light:

```
Red night light   fw drive R 0.0121            -> LEDs lit WW 0.0121, G 0.0121, R 0
(violates R20)
Soft orange night fw drive WW 0.0014 R 0.0174   -> LEDs lit WW 0.0188, G 0.0174, CW
0.0027, R 0.0014
Warm white 2700K  fw drive WW 0.1099            -> LEDs lit WW 0.11, CW 0.22, R 0.11
Green             fw drive G                    -> LEDs lit CW + B       Blue -> WW + CW
```

  OUT14/OUT15 (D10 G/B) were never enabled, and the unconnected OUT4/OUT5 were enabled.
- **Status.** At 17:28 `hardware/build.py` gained `write_board_map(d.out_group())` and regenerated the header. The new array equals the PCB pad nets and the 17:28 schematic netlist (re-verified with kicad-cli). The scratch F01 tests now pass on the repo.
- **Residual (MINOR).** Nothing checks the *routed PCB* against `board_map.h`. `out_group()` is recomputed from placement, so if the board is finished by hand in KiCad (allowed by the repo's CLAUDE.md), the map can drift again.
    - Suggested fix: after `layout.build_pcb`, read the U1 pad → `K_*` nets from the `.kicad_pcb` and `assert` they equal `d.out_group()` (the extraction in `fwtest/` does exactly this with a 10-line regex).
    - Optionally, emit the pad→net table into `board_map.h` as a comment.

FW-02: blink window and fade use absolute deadlines that wrap: a 5 Hz full on/off strobe after 24.86 days

- **Where:** `lib/core/lightcore.cpp:130` (`blink_until = now + 400`), `:196` (`now + 200`), `:202` (`(int32_t)(c->blink_until - now) > 0`), `:199-201` (fade `k` from `now - fade_start`); `lightcore.h:62`.
- **Issue.**
    - `core_init` memsets `blink_until = 0` and `blink_level = 0`. Once `now - blink_until` exceeds 2^31 ms (24.86 days after power-on, or after the last save or low-battery blink), the signed test becomes true again for the next 24.86 days.
        - `left` is then huge. With `blink_level` 0 (< 0.2), `dip = (left/100)%2 == 1` toggles every 100 ms and multiplies the drive by 0.
        - Result: a **full-depth 5 Hz strobe**. That violates R19 "flicker-free" and falls in the photosensitive 3–30 Hz band, in a lamp used as a night light.
        - After a save, the same thing happens between 100 % and 15 %. After a low-battery blink, the lamp sits at 25 % for 24.86 days.
    - Separately, `now - fade_start` wraps after 2^32 ms. `k` restarts at 0 and the light jumps back to `fade_from`, which is 0 after power-on, then re-fades over 160 ms.
- **Precondition.** ≥ 24.86 days of continuous on-time. On battery the cell empties first (a reset clears `millis()`), so this happens on USB power with the slide switch left on: a plausible bedside-night-light use.
- **Evidence.**
    - `run_fulltick` ticks `core_buttons` + `core_tick` every ms from power-on, with no jump: `steady drive 0.2057; in the 3 s after 2147483648 ms (24.86 d): min 0.0000, 30 light/dark transitions`.
    - `test_F03_no_strobe_after_24_8_days_on`: 20 transitions in 2 s.
    - `test_F03_no_refade_after_49_7_days_on`: min drive 0.0013 against 0.2057.

- **Patch** (in `firmware-review.patch`). Store `blink_start` + `uint16_t blink_len` and test only the elapsed time (`now - blink_start < blink_len`, then clear `blink_len`). Add `fade_active`, cleared when the fade completes, at which point `fade_from = target`, so a later wrap re-fades target→target (no change):

```
        -    uint32_t blink_until;             // confirmation/low-battery blink window
        +    bool fade_active;                 // cleared when the fade completes, so a
millis() wrap cannot restart it
        +    uint32_t blink_start;             // confirmation/low-battery blink window:
start ...
        +    uint16_t blink_len;               // ... and length in ms (0 = no blink; no
absolute deadline to wrap)
        ...
        -    float k = (float)(now - c->fade_start) / (float)FADE_MS;
        -    if (k > 1.0f) k = 1.0f;
        +    float k = 1.0f;
        +    if (c->fade_active) {
        +        uint32_t el = now - c->fade_start;
        +        if (el >= FADE_MS) { c->fade_active = false; for (int g = 0; g <
G_COUNT; g++) c->fade_from[g] = c->target[g]; }
        +        else k = (float)el / (float)FADE_MS;
        +    }
        ...
        -    if ((int32_t)(c->blink_until - now) > 0) {
        -        uint32_t left = c->blink_until - now;
        +    if (c->blink_len) {
        +        uint32_t el = now - c->blink_start;          // elapsed time only: wrap-
safe for any uptime
        +        if (el >= c->blink_len) c->blink_len = 0; else { uint32_t left = c-
>blink_len - el; ...
```

  The dip pattern is unchanged (two 100 ms dips in 400 ms; one 200 ms dip). The repo's 17 tests still pass. Add the two wrap tests from `fwtest/test_review.cpp` to `test/test_core/`.

MAJOR

FW-03: `modes_table.h` was generated for the wrong RGB LED and the wrong current; candle 1900 K is yellow-green

- **Where:** `tools/gen_modes.py:19-25`; `include/modes_table.h:17-18`.
- **Issue.** `gen_modes.py` uses Kingbright KAAF-5050RGBS photometry (R/G/B = 4.4/4.4/1.3 lm per LED) and white flux at 30 mA (12/13.5 lm).
  - PLAN's decision log switched the RGB to the **Würth 150141M173100**. KEYSPECS §2 gives Iv typ at 20 mA of R 270 mcd, G 950 mcd, B 230 mcd, i.e. about 1.06 / 3.73 / 0.90 lm at 25 mA (π·Iv × 25/20).
  - The channels run at 25.0 mA. Bridgelux DS312 Fig. 10 gives about 20 % of the 150 mA flux at 30 mA (KEYSPECS §3), i.e. about 8.5 / 9.3 lm at 25 mA.
  - Red is about 4× weaker than assumed, so the mixes that lean on red are wrong (R20: "whites on the Planckian locus").
- **Evidence.** `fwtest/py/modes_check.py` re-evaluates the shipped table with the fitted parts:

Mode	Designed xy	xy with fitted LEDs
Candlelight 1900 K	(0.5389, 0.4103)	**(0.4104, 0.5019)**, Δxy 0.158: a yellow-green
Soft orange night	(0.6429, 0.3256)	(0.5863, 0.3519), Δxy 0.062

White modes move by Δxy ≤ 0.002. Single-channel modes are unaffected.

- **Patch.** Update the `LED` table and regenerate (part of `firmware-review.patch`):

```
        -    "R": (0.700, 0.299, 4.4, 2),       # 2 x Kingbright KAAF-5050RGBS 625 nm,
1.4 cd -> ~4.4 lm each (Lambertian, pi*Iv)
        -    "G": (0.170, 0.700, 4.4, 2),       # 525 nm, 1.4 cd
        -    "B": (0.131, 0.075, 1.3, 2),       # 470 nm, 0.42 cd
        +    "R": (0.700, 0.299, 1.06, 2),      # 2 x Wurth 150141M173100 red 625 nm: 270
mcd typ @20 mA -> pi*Iv x 25/20
        +    "G": (0.170, 0.700, 3.73, 2),      # green 520 nm: 950 mcd typ @20 mA (xy
VERIFY: 520 nm sits greener than this)
        +    "B": (0.131, 0.075, 0.90, 2),      # blue 470 nm: 230 mcd typ @20 mA
```

  The white rows change to 8.5 / 9.3 lm.

  - Regenerated values: candle `{86, 0, 4096, 1046, 0}`, which checks to (0.5392, 0.4103); orange `{109, 0, 4096, 0, 0}`.
  - **Consequence for Matt:** with the weak Würth red, candle tops out at about **4.7 lm** and red night at about **2.1 lm** at full drive. Mark the LED photometry and the G chromaticity as VERIFY items and re-run with measured values.

FW-04: `I_CHANNEL_MA` does not match REXT

- **Where:** `lib/core/lightcore.h:21`: `#define I_CHANNEL_MA 30.2f // ... (REXT 620 R, CG 0.992)`.
- **Evidence.** The LED board has R1 = 750 Ω (design.py:125, parts.py:96). TLC59116 eq. (5): IOUT = (1.26 V / Rext × 15) × CG = 1.26/750 × 15 × 0.992 = **25.0 mA**. The spec requires the constant to match. `test_F02_i_channel_matches_rext_750` fails: 25.00 vs 30.20.
- **Effect.** It is safe in direction (it over-estimates current), but every current figure is 21 % high.
  - 4000 K at level 15 computes 220 mA, so it is capped to 200/220. Real current is then **165.5 mA**, although the budget allows the uncapped 182 mA. The light is about 9 % dimmer than intended in that mode.
  - Note also that the 200 mA "~0.6 W" budget is LED-board dissipation at VLED: 0.2 A × 4.15 V ≈ 0.83 W on USB, because the TLC drops VLED − VF.
- **Patch:**

```
        -#define I_CHANNEL_MA      30.2f     // TLC59116 full-scale current (REXT 620 R,
CG 0.992)
        +#define I_CHANNEL_MA      25.0f     // TLC59116 full-scale current: 1.26 V / 750
R x 15 x CG 0.992 (led-board R1)
```

  Also fix the "30 mA" comments in `gen_modes.py`.

FW-05: LM75B stays in shutdown after a resume from battery-empty, freezing thermal derating

- **Where:** `src/main.cpp:118-121` writes Conf = 0x01 (shutdown) before power-down. `:135` does a software reset. `setup()` (`:145-163`) never writes Conf back to 0.

- **Evidence (LM75B §7.1):** "In shutdown mode, the device becomes idle, data conversion is disabled and the Temp register holds the latest result … The temperature conversion is initiated when the device is powered-up or put back into normal mode from shutdown."
  - On the USB-resume path VSYS never drops, so the LM75B keeps Conf = 0x01.
  - `lm75_read` still ACKs and returns the frozen pre-sleep temperature with `ok = true`. Neither the thermal derating (R19) nor the 3 s sensor failsafe ever reacts until the slide switch is cycled.
- **Patch.** Add `lm75_config(uint8_t)` and call `lm75_config(0x00)` in `setup()` after `Wire.begin()`. The first read happens at 1 s, after the 100 ms conversion. Also reject implausible readings (FW-16).

FW-06: no watchdog

- **Where:** `src/main.cpp` (no WDT anywhere). The factory/PlatformIO fuse is WDTCFG = 0x00.
- **Issue.** Any hang (an unforeseen loop, or a bus state the 10,000-poll Wire timeout does not cover) freezes the TLC59116 at its last PWM/IREF.
  - The LEDs stay lit at up to 182 mA with no thermal derating (R19) and no empty-battery cut-off. Only the slide switch recovers.
  - The TLC keeps its registers independently of the MCU (§9.3.3: POR only below 0.2 V).
- **Patch** (validated build).
  - Set `_PROTECTED_WRITE(WDT.CTRLA, WDT_PERIOD_2KCLK_gc)` (2 s) in `setup()`, and `__builtin_avr_wdr()` at the end of each `loop()`.
  - Also `wdr` after every PIT wake in `sleep_until_resumed()`. Datasheet §23: "When enabled, the WDT will run in Active mode and all sleep modes", so the 1 s PIT keeps it fed.
  - The longest blocking paths are an EEPROM put (≤ ~20 ms) and a Wire timeout (~10–20 ms), both far below 2 s.
  - megaTinyCore copies RSTFR to GPIOR0 at boot, so WDT resets can be counted or logged in the debug build.
  - Pair with FW-15 (bus recovery), because a WDT reset can land mid-transfer.

FW-07: TLC59116 error handling and recovery

- **Where:** `src/main.cpp:24-41` (`tlc_init`), `:68-73` (IREF/PWM writes), `:189-193` (loop).
- **Issues:**
  1. In `tlc_init`, only the MODE write's result is checked; the PWM, GRP, LEDOUT and IREF results are ignored, and it returns `true`. A NACK on the LEDOUT write leaves all LDR = 00 (off) forever.
  2. `tlc_ok` is never cleared after init. If the LED board browns out or is re-plugged, the TLC resets to MODE1 = 11h (OSC off) and LEDOUT = 00 ("LDRx = 00: LED driver x is off (default power-up state)"). The firmware keeps writing PWM, which now ACKs, and the moon stays **dark with the slide switch on** until power is cycled.
  3. `iref_code` is updated before the IREF write, and the write's result is ignored. One NACK leaves firmware and chip disagreeing about the gain indefinitely: PWM is computed for the wrong CG (2× per step).
- **Evidence** (`fwtest/test_review.cpp`, register model per SLDS157E):

```
F06 after one failed IREF write: chip IREF 0x3F, fw iref_code 0x00, OUT2 wanted 0.00400
got 0.00781 (x1.95)
F07 5 s after a TLC POR: MODE1 0x11 LEDOUT0 0x00, total output 0.0000 (drive sum 0.6533)
F07 LEDOUT write NACKed once during tlc_init(); tlc_ok 1, total output after 2 s 0.0000
```

- **Patch** (validated: all three pass).
  - `tlc_init` returns the AND of all writes.
  - `tlc_output` returns `false` on any failed write, updates `iref_code` only after a successful IREF write, and the loop sets `tlc_ok = false` (re-init after 500 ms).
  - Once a second the loop reads MODE1 back and re-inits if OSC or ALLCALL is set:

```
        if (tlc_ok && now - last_tlc_check >= 1000) {   // LED board POR (cable,
VLED dropout) resets MODE1 to 11h
            last_tlc_check = now;
            int16_t m1 = tlc_read(0x00);
            if (m1 < 0 || (m1 & 0x11)) tlc_ok = false;  // OSC or ALLCALL set = not
our configuration -> re-init
        }
```

  - Single-register writes use control byte `reg` without AI (N-2).

FW-08: VDD measurement: first conversion after the reference starts is averaged in; resume uses one raw sample

- **Where:** `src/main.cpp:96-111` (`vdd_mv`), `:132` (`if (vdd_mv() >= VBAT_RESUME_MV) break;`).
- **Register set-up is correct:**
  - `VREF.CTRLA.ADC0REFSEL = 1V1`; `CTRLC = SAMPCAP | REFSEL_VDD | DIV16` (CLK_ADC 625 kHz, inside 200–1500 kHz); `SAMPCTRL 16` (Ts = 18 CLK_ADC ≤ 33); `MUXPOS = INTREF (0x1D)`.
  - The formula matches §30.3.2.5, RES = 1023·VIN/VREF.
  - The 1.1 V reference is valid for VDD 1.8–5.5 V (Table 36-22), so 3.0 V is fine.
  - millis runs on TCD0 (`-DMILLIS_USE_TIMERD0`), so the ADC and RTC are free.
- **Issue 1: first sample.** VREF §18.2: "A voltage reference source is enabled automatically when requested by a peripheral." Its start-up is 25 µs typ (Table 36-22), with no max given. INITDLY applies only "when enabling the ADC or changing to an internal reference voltage" (CTRLD); here the ADC is already enabled and REFSEL stays VDD.
  - megaTinyCore's own `readSupplyVoltage()` (libraries/megaTinyCore/src/megaTinyCore.cpp:66-71) says: "since the reference is normally turned on only when it is requested, this virtually guarantees that the first reading will be garbage", and discards it.
  - `vdd_mv()` averages it into 8 samples. Worst case (first sample ≈ 0), RES reads 7/8 of true, so **VDD reads +14 %**.
    - Empty (3.30 V reading) then corresponds to about 2.9 V real.
    - A resting "empty" cell at 3.4 V reads 3.89 V, which is above `VBAT_RESUME_MV` 3700. Resume uses that single unfiltered sample, so a false resume → reboot → ~9 s on → empty → sleep loop could repeat every few seconds until the PCM trips.
  - Bench-verify; the fix is cheap regardless.
- **Issue 2: accuracy.** INT11V is ±2.0 % (VDD 1.8–3.6 V, 0–105 °C) and ±5.0 % over the full range (Table 36-23, "characterization, not production tested"), plus about 3 LSB. That is about ±100 mV at 3.3 V, uncalibrated, so the thresholds 3300/3450/3700 move by that much.
- **Patch.**
  - Set `VREF.CTRLB |= ADC0REFEN` for the burst, `delayMicroseconds(50)`, discard conversion 0, then average 8.
  - Resume only after 3 consecutive 1 s readings ≥ 3700.
  - Optionally store a per-unit calibration factor (measured VDD / reading) in USERROW.

FW-09: fuses (BOD, EESAVE, SUT) are never programmed by the documented flash command

- **Where:** `platformio.ini:2` ("Flash: `pio run -t upload`"), `:13-14`.
- **Evidence.**
  - PlatformIO `atmelmegaavr/builder/main.py:218-228` appends FUSESFLAGS to upload only for `uno_wifi_rev2` and `nano_every`.
  - For ATtiny3216 the fuses are written only by `pio run -t fuses` (`fuses.py` computes BODCFG 0x54 = 2.6 V enabled-active / off-in-sleep, SYSCFG0 EESAVE = 1, SYSCFG1 0x06 = 32 ms).
  - Factory BODCFG reset value is 0x00, i.e. **BOD disabled** (datasheet §6.10.4.2).
  - Table 36-3: 10 MHz is specified at VDD ≥ 2.7 V, "Operation ensured down to … VBOD with BODLEVEL2". So the 2.6 V choice is correct *if programmed*. Without it, slow ramps and sags (slide switch, USB↔battery hand-over, PCM cut-off at 2.4–3.0 V) run the core out of spec.
  - EESAVE = 0 means every reflash erases the saved default.
- **Fix.** Document and run `pio run -e attiny3216 -t fuses` once per board, or `-t fuses -t upload`. Fix the ini comment.
  - Add a boot check in the debug build: `if ((FUSE.BODCFG & 0x0C) == 0) Serial.println(F("BOD OFF: run pio run -t fuses"));`.
  - BOD.CTRLB LVL is loaded from the fuse, so firmware cannot enable it by itself.

FW-10 (system): the empty-battery "sleep" leaves the LED board on VSYS; TLC59116 standby current is unspecified

- **Where:** `src/main.cpp:76-81`, `:116-136`; the hardware puts VLED = VSYS, with no MCU-controlled switch.
- **Evidence.**
  - TLC59116 §7.5 lists only **maximum** ICC at VCC 5.5 V: 25 mA (outputs OFF, Rext open) and 29 mA (outputs OFF, 720 Ω). There is no typical and no figure for MODE1.OSC = 1.
  - PLAN decision log: "Cost: 25–37 mA driver supply current while on".
  - MCU power-down + PIT is about 2 µA max and LM75B shutdown 1 µA max, so the sleep current is whatever the TLC draws with OSC off.
  - If that is in the mA range, a 1200 mAh LiPo at "empty" (about 5–10 % left) reaches PCM cut-off within hours. That repeats a deep discharge every time the light is left on.
- **Action.** Measure VSYS current in `sleep_until_resumed()` on the prototype (VERIFY item).
  - If it is above about 50 µA, add a VLED load switch driven by a spare MCU pin (PA7/PB2…) or reconsider the driver.
  - Firmware cannot fix this beyond OSC = 1 and PWM = 0, which it already does.

MINOR

FW-11: IREF/PWM write order gives a 2× flash at each gain step up (R19)

- **Where:** `src/main.cpp:68-73`.
- **Evidence.** MODE2 Table 5 note 3: outputs change at STOP, "Applicable to registers from 02h (PWM0) to 17h (LEDOUT3) only". So IREF (1Ch) takes effect on its own write, about 1.8 ms before the 18-byte PWM write's STOP.

    - IREF is always written first, so on a gain *increase* the LEDs briefly run new-gain × old-PWM.
    - The register model sweeps levels 15→0→15: `worst in-between/endpoint brightness ratio on an IREF step: 1.98x`.
    - With AI = 100, a single transaction cannot reach 1Ch: Table 14 "roll over to 0 0000 after the last register (1 1011)".
- **Patch** (validated: 1.00×). Gain up: write PWM first, then IREF. Gain down: write IREF first, then PWM. The in-between state is then never brighter than either end.

FW-12: PIT wake depends on an RTC erratum on Rev A silicon

- **Where:** `src/main.cpp:122-126`. It sets CLKSEL and enables the PIT with RTC.CTRLA.RTCEN = 0.
- **Evidence.** megaTinyCore `extras/Errata.md`, "Disabling the RTC Stops the PIT (0/1-series)". Its example sequence ("Set RTC clock source but don't enable it. Enable the PIT… Observe … absence of behavior relating to the PIT") is exactly this code. The same file says the 32 KB 1-series "got a Rev. C that fixed the RTC bug".
    - On Rev A silicon the lamp may never wake to see USB power; the user then has to cycle the switch. The datasheet sequence (§23.5.1) itself is followed correctly.
- **Action.** Read `SYSCFG.REVID` on the prototype and bench-test the USB-resume path. If Rev A, also enable the RTC (`RTC.CTRLA = RTC_RTCEN_bm`) before the PIT.

FW-13: `static uint32_t last_slew` inside `core_tick`

- **Where:** `lib/core/lightcore.cpp:179`.
- **Issue.** The state is shared by all `core_t` instances and leaks between host tests: each test's first tick sees a wrapped `now - last_slew` and a clamped 0.01 step.
- **Evidence.** `test_F04_thermal_slew_independent_per_instance` ticks two instances at the same `now: a t_cap 0.050, b t_cap 1.000`.
- **Patch.** Move it into `core_t` and initialise it in `core_init`. Validated.

FW-14: CPU budget

- **Measured** (simavr, avr5 ≈ AVRxt):
    - `core_tick` takes 13.4 k cycles = **1.34 ms at 10 MHz while the thermal cap slews**. It calls `mode_targets`, and so `powf`, every ms.
    - `tlc_output` spends 12.9 k cycles on maths every 4 ms, even when nothing changed, plus about 1.75 ms of I2C when it did.
- **Model** (`fwtest/py/loop_model.py`). While slewing the loop never idles and `(now & 3)==0` is missed: 181 updates/s instead of 250, gaps up to 12 ms.
- **Effect.** Cosmetic today, but it leaves no margin.
- **Fix.**
    - Replace `level_fraction()` with a 16-entry table (`powf` is 6.3 k cycles).
    - Skip `tlc_output` when `drive[]` is unchanged.
    - Consider Q15 fixed point for the PWM computation.
    - Consider 400 kHz I2C: TLC FM+ and LM75B are both 400 kHz-capable, but check the cable rise time with 3.3 kΩ.

FW-15: no I2C bus recovery

- **Where:** `src/main.cpp:156`.

- **Issue.** The TLC59116 has no bus timeout (the LM75B has one, 75 ms per §7.2.1). A reset mid-ACK leaves SDA low. megaTinyCore's `Wire.begin()` only forces BUSSTATE to IDLE, and each transfer then times out after F_CPU/1000 polls.
- **Patch.** `i2c_bus_recover()` before `Wire.begin()`: up to 9 SCL pulses while SDA is low, then a STOP, open-drain via `pinMode`. Validated build.

FW-16: LM75B byte assembly and plausibility

- **Where:** `src/main.cpp:89`, `int16_t raw = (int16_t)((Wire.read() << 8) | Wire.read());`.
- **Issue.** Operand evaluation order of `|` is unspecified in C++.
  - avr-gcc 7.3 happens to call the left `read()` first: disassembly `firmware.elf` 0xfbe/0xfc8, first result kept in r17 → high byte.
  - A compiler or LTO change could swap the bytes: 25.0 °C (0x1900) would read 0.0 °C and disable derating.
  - The pointer (00h), the 11-bit two's complement `>>5` (arithmetic `asr`, verified) and 0.125 °C/LSB are correct, as is the shutdown write format.
- **Patch.** Use two statements. Return `false` outside −55…125 °C.

FW-17: uninitialised float passed when the read fails

- **Where:** `src/main.cpp:175-177` (`-Wmaybe-uninitialized`, hidden by `-w`).
- **Issue.** `core_temperature` ignores it when `!ok`, but reading an indeterminate value is UB.
- **Patch.** `float t = 0.0f;`.

FW-18: build hygiene and test coverage

- **Warnings are hidden.** The framework appends `-w` *after* `-Wall -Wextra` (see the verbose build line), so the AVR build shows no warnings at all. Fix: `build_unflags = -w`.
  - That exposes a stale PlatformIO-builder define, `-DUARTBAUD5V`, which raises a `#warning` in every translation unit ("removed in 2.3.0 … does nothing"). The patched project's own sources compile warning-free.
- **The native env has no warning flags.** Add `-Wall -Wextra` (in the patch; clean).
- **Coverage.** `test_build_src = no`, so none of `main.cpp` is host-tested: gain/PWM selection, LM75 conversion, VDD maths. Move those pure functions into `lib/core` (or a second portable unit) and test them.
- **Int width.** The host `int` is 32-bit and AVR's is 16-bit. Any `uint16_t*3u` style arithmetic (lightcore.cpp:158 is safe today) is only exercised on 32-bit. Use explicit `uint32_t` casts.
- **Weak tests.**
  - `test_save_confirmation_dips` computes `during` and never asserts it.
  - `test_battery_low_empty_resume` uses 4700 mV for USB; the real value is ~4150 mV.

FW-19: soft-orange night mode includes the phosphor white (owner decision against R20's wording)

- **Where:** `include/modes_table.h:18`.
- **Issue.** The G/B channels are 0 (repo test plus the scratch test on the real map), but 30 % of the luminance is the 2700 K phosphor-converted white. Its spectrum has a ~450 nm blue pump peak and green phosphor emission.
  - PLAN R20 says "Night modes contain no green or blue light". `docs/research/light-modes.md` deliberately chose "red plus a little 2700 K, never green" (mel DER 0.14).
- **Action.** Confirm the interpretation with Matt (channels vs spectrum), or make orange R-only with a different name.

NOTES (no action required unless stated)

- **N-1 PWM full scale.** Duty = IDC/256 (eq. 1), so `drive 1.0 → 255 = 99.6 %`. `p = v·255` under-drives by 0.4 % at all levels. Use `v·256` clamped to 255, or LDR = 01 at exactly 1.0. Negligible.
- **N-2 Single-byte writes.** The IREF write uses control byte 0x9C (AI2 set, address 1Ch, beyond the AI = 100 range 00h–1Bh). It is harmless for one byte, but plain 0x1C avoids undefined auto-increment. Patched.
- **N-3 IREF bit order.** The datasheet contradicts itself: Table 12 puts CM at bit 7 and HC at bit 6, while the prose says "Bit 0 is the Current Multiplier (CM) bit" with `CC[0:5]` notation.
  - The firmware uses only CC = 0 or 63, which is immune to CC bit order. The CM/HC positions still need a bench check: measure OUT current at 0x7F vs 0xBF; expect 8.3 vs 12.5 mA at 750 Ω.
  - The CG table values themselves (0.0833/0.1654/0.3307/0.4961/0.9922) are correct for {CM,HC,CC} = 000/0x3F/0x7F/0xBF/0xFF.
- **N-4 Low-end accuracy.**
  - Gain steps 0x3F (4.2 mA) and 0x00 (2.1 mA) are below the recommended IO 5–120 mA ("CM = 0 … suitable for … 5 mA to 40 mA"). Matching (±6 % spec) is only guaranteed at 26 mA.
  - At levels 0–2 the minor channels quantise to 1–2 PWM counts. Worst ratio error: 3000 K 19 %, candle 33 %, orange 28 % (`fwtest/py/quant.py`).
  - Expect a tint shift at the lowest levels. A finer CG ladder (any CC value) would help.
- **N-5 Battery policy.**
  - `LOW_CAP` scales the level (`f = level_fraction × cap`) rather than capping it, so a night light at level 3 also drops to 35 % when low.
  - The 100 mV low/ok hysteresis is less than ΔI·R once series resistance exceeds about 0.85 Ω (ΔI ≈ 118 mA between full and 35 %). Consider latching "low" until USB or a power cycle.
  - USB VSYS of about 4.15 V clears low/empty correctly (3700 resume). The empty hold is 5 s and the IIR τ is about 1.7 s.
- **N-6 Boot.**
  - From power-on, the TLC POR leaves LDRx = 00 (off), so the moon is dark until `tlc_init()`. Boot takes the 32/64 ms SUT, not 5 ms.
  - The OSC → 600 µs delay → PWM ordering meets the MODE1 note (500 µs).
  - After an MCU-only reset (BOD/SWRST/WDT), the TLC keeps the last image until `tlc_init` zeroes it and then fades in, which is a visible blink.
  - The `delay(5)` is redundant (same rail) but harmless.
- **N-7 EEPROM.**
  - `EEPROM.put` is per-byte `update()` (EEPROM.h:261-265), so there is no wear issue.
  - Magic + version + range + CRC-8 reject blank (0xFF) and corrupt data, falling back to 3000 K / level 10.
  - The loop stalls up to about 4 ms per changed byte.
  - `saved_valid` accepts a level above `MODES[mode].max_level`; it is clamped on output, but the first DOWN presses then do nothing. Clamp in `core_init`.
- **N-8 Timing.**
  - millis runs on TCD0: 20 MHz/32, TOP 0x1FD, ISR every 0.816 ms, never skips a value. So the RTC/PIT are free: no millis conflict.
  - Idle sleep wakes on that ISR. All other interval checks are wrap-safe unsigned differences.

    - I2C load: about 45 % of the bus at 100 kHz during fades, 0 in steady state, LM75 at 1 Hz.
- **N-9 Core APIs verified.**
    - `Serial.swap(1)` puts USART0 TX on PA1 / RX on PA2. `digitalReadFast` exists, needs LTO (on) and returns non-zero.
    - Wire default pins are PB0 SCL / PB1 SDA. The TWI buffer is 32 bytes (RAMSIZE < 4096), enough for 17 bytes, and the patch's 28.
    - `_PROTECTED_WRITE(RSTCTRL.SWRR, SWRE)` is correct. PIT `CYC32768` from INT32K gives 1 s, and the PIT runs in power-down ("Only the PIT is available").
    - Button pins PA6/PA5/PA4 match the main-board netlist; TX is PA1 (pin 17).
- **N-10 G/B headroom.** Würth G/B VF is about 3.27–3.40 V at 30 mA, plus the TLC knee of about 0.3 V. They fall out of regulation below VLED ≈ 3.6–3.7 V, so G-containing whites (3000/4000/6500 K) and candle drift on battery before "low" (3.45 V).
- **N-11 State machine vs spec, verified.**
    - 15 ms debounce; MAIN cycles on release; hold ≥ 2000 ms saves mode + level once, with two 100 ms dips, and release does not cycle.
    - UP/DOWN repeat after 450 ms, then every 150 ms, clamped 0…max_level. Per-mode level memory.
    - No path turns the light off except the slide switch and the protective battery-empty fade (R5, R7).
    - The current cap holds for every mode and level.
- **N-12 PlatformIO env.** `board = ATtiny3216`, `f_cpu 10000000L` + internal (OSCCFG 20 MHz/2), `upload_protocol = serialupdi` @230400 and `lib_deps = core` are correct; `upload_port` is unset (auto-detect). The native env's `-lm` in `build_flags` works.

Reproduce

```
cd /home/user/reading_light/firmware && pio test -e native && pio run -e attiny3216      # repo
baseline
/tmp/claude-0/-home-user/7ba8e87c-d4ee-5a66-9891-51bde4e0a9a7/scratchpad/review/fwtest/run.sh
# scratch suite: repo, then patched copy
/tmp/claude-0/-home-user/7ba8e87c-d4ee-5a66-9891-
51bde4e0a9a7/scratchpad/review/fwtest/run_fulltick      # 24.86-day strobe, no time jump (~25
s)
python3 -I /tmp/claude-0/-home-user/7ba8e87c-d4ee-5a66-9891-
51bde4e0a9a7/scratchpad/review/fwtest/py/modes_check.py
/home/user/reading_light/firmware/include/modes_table.h
simavr -m atmega328p -f 16000000 /tmp/claude-0/-home-user/7ba8e87c-d4ee-5a66-9891-
51bde4e0a9a7/scratchpad/review/fwtest/cycles/cyc.elf
```

The patched tree is at `/tmp/claude-0/-home-user/7ba8e87c-d4ee-5a66-9891-51bde4e0a9a7/scratchpad/review/fwtest/patched_fw/`.

Appendix: full proposed patch (`review/patches/firmware-review.patch`)

The patch is against the reviewed snapshot. `board_map.h` is excluded because the working tree already holds the regenerated map. `modes_table.h` is shown as `gen_modes.py` regenerates it. Per the repo's CLAUDE.md, apply the `gen_modes.py` change and run the generator rather than hand-editing the header.

```
        --- a/firmware/lib/core/lightcore.h
        +++ b/firmware/lib/core/lightcore.h
        @@ -18,7 +18,7 @@
         #define REPEAT_EVERY_MS   150
         #define DEBOUNCE_MS       15
         #define FADE_MS           160       // cross-fade on any change
        -#define I_CHANNEL_MA      30.2f     // TLC59116 full-scale current (REXT 620 R, CG
0.992)
        +#define I_CHANNEL_MA      25.0f     // TLC59116 full-scale current: 1.26 V / 750 R x
15 x CG 0.992 (led-board R1)
         #define I_CAP_MA          200.0f    // total LED current cap (moon-base thermal
budget, ~0.6 W)
         #define T_DERATE_START_C  50.0f     // LED board temperature where derating starts
         #define T_DERATE_KNEE_C   65.0f     // 30 % at the knee
        @@ -59,9 +59,12 @@
             float target[G_COUNT];
             float fade_from[G_COUNT];
             uint32_t fade_start;
        -    uint32_t blink_until;             // confirmation/low-battery blink window
        +    bool fade_active;                 // cleared when the fade completes, so a
millis() wrap cannot restart it
        +    uint32_t blink_start;             // confirmation/low-battery blink window: start
...
        +    uint16_t blink_len;               // ... and length in ms (0 = no blink; no
absolute deadline to wrap)
             float blink_level;
             bool save_request;                // main loop writes EEPROM when set
        +    uint32_t last_slew;               // thermal slew timestamp (per instance, not a
function static)
         } core_t;

         void core_init(core_t *c, const saved_t *saved, uint32_t now);
        --- a/firmware/lib/core/lightcore.cpp
        +++ b/firmware/lib/core/lightcore.cpp
        @@ -72,6 +72,7 @@
             for (int g = 0; g < G_COUNT; g++) c->fade_from[g] = c->drive[g];
             mode_targets(c->mode, c->level[c->mode], limits(c), c->target);
             c->fade_start = now;
        +    c->fade_active = true;
         }

         void core_init(core_t *c, const saved_t *saved, uint32_t now) {
        @@ -87,6 +88,7 @@
             c->t_cap = c->t_cap_target = 1.0f;
             c->t_last_ok = now;
             c->vbat_mv = 4000;
        +    c->last_slew = now;
             retarget(c, now);   // fades in from 0 on power-up
         }

        @@ -127,7 +129,8 @@
                         if (!c->main_save_done && held >= HOLD_SAVE_MS) {
                             c->main_save_done = true;
                             c->save_request = true;
        -                    c->blink_until = now + 400;          // confirmation: two dips
(see core_tick)
        +                    c->blink_start = now;                // confirmation: two dips
(see core_tick)
        +                    c->blink_len = 400;
```

```
                            c->blink_level = 0.15f;
                        }
                    } else if (held >= REPEAT_DELAY_MS && (uint32_t)(now - c->last_repeat[b])
>= REPEAT_EVERY_MS) {
        @@ -176,34 +179,46 @@

         void core_tick(core_t *c, uint32_t now) {
             // thermal cap slews at ~10 %/s (0.0001 per 1 ms tick) so derating is gentle and
invisible
        -    static uint32_t last_slew;
             float d = c->t_cap_target - c->t_cap;
        -    if ((d > 0.0005f || d < -0.0005f) && now != last_slew) {
        -        float step = (float)(now - last_slew) * 0.0001f;
        +    if ((d > 0.0005f || d < -0.0005f) && now != c->last_slew) {
        +        float step = (float)(now - c->last_slew) * 0.0001f;
                 if (step > 0.01f) step = 0.01f;
                 if (d < 0) step = -step;
                 if ((d > 0 && step > d) || (d < 0 && step < d)) step = d;
                 c->t_cap += step;
        -        bool fading = (uint32_t)(now - c->fade_start) < FADE_MS;
                 mode_targets(c->mode, c->level[c->mode], limits(c), c->target);
        -        if (!fading)
        +        if (!c->fade_active)
                     for (int g = 0; g < G_COUNT; g++) c->fade_from[g] = c->target[g];
             }
        -    last_slew = now;
        +    c->last_slew = now;
             // low-battery reminder blink
             if (c->batt == 1 && (uint32_t)(now - c->last_low_blink) >= LOW_BLINK_EVERY_MS) {
                 c->last_low_blink = now;
        -        c->blink_until = now + 200;
        +        c->blink_start = now;
        +        c->blink_len = 200;
                 c->blink_level = 0.25f;
             }
        -    float k = (float)(now - c->fade_start) / (float)FADE_MS;
        -    if (k > 1.0f) k = 1.0f;
        +    float k = 1.0f;
        +    if (c->fade_active) {
        +        uint32_t el = now - c->fade_start;
        +        if (el >= FADE_MS) {
        +            c->fade_active = false;
        +            for (int g = 0; g < G_COUNT; g++) c->fade_from[g] = c->target[g];
        +        } else {
        +            k = (float)el / (float)FADE_MS;
        +        }
        +    }
             for (int g = 0; g < G_COUNT; g++) c->drive[g] = c->fade_from[g] + (c->target[g] -
c->fade_from[g]) * k;
        -    if ((int32_t)(c->blink_until - now) > 0) {
        -        // dips: for the save confirmation two 100 ms dips inside 400 ms; for low
battery one 200 ms dip
        -        uint32_t left = c->blink_until - now;
        -        bool dip = (c->blink_level < 0.2f) ? ((left / 100u) % 2u == 1u) : true;
        -        if (dip)
        -            for (int g = 0; g < G_COUNT; g++) c->drive[g] *= c->blink_level;
        +    if (c->blink_len) {
        +        uint32_t el = now - c->blink_start;          // elapsed time only: wrap-safe
for any uptime
        +        if (el >= c->blink_len) {
```

```
        +            c->blink_len = 0;
        +        } else {
        +            // dips: for the save confirmation two 100 ms dips inside 400 ms; for low
battery one 200 ms dip
        +            uint32_t left = c->blink_len - el;
        +            bool dip = (c->blink_level < 0.2f) ? ((left / 100u) % 2u == 1u) : true;
        +            if (dip)
        +                for (int g = 0; g < G_COUNT; g++) c->drive[g] *= c->blink_level;
        +        }
             }
         }
        --- a/firmware/src/main.cpp
        +++ b/firmware/src/main.cpp
        @@ -16,35 +16,45 @@
         // ---------------------------------------------------------------- TLC59116
         static bool tlc_write(uint8_t reg, const uint8_t *d, uint8_t n) {
             Wire.beginTransmission(ADDR_TLC);
        -    Wire.write((uint8_t)(0x80 | reg));          // AI2=1: auto-increment through all
registers
        +    // AI=100 (all registers, rolls over after 1Bh) only for block writes; single
registers (IREF 1Ch) without AI
        +    Wire.write((uint8_t)(n > 1 ? (0x80 | reg) : reg));
             for (uint8_t i = 0; i < n; i++) Wire.write(d[i]);
             return Wire.endTransmission() == 0;
         }

        +static int16_t tlc_read(uint8_t reg) {          // -1 on bus error
        +    Wire.beginTransmission(ADDR_TLC);
        +    Wire.write(reg);                             // control byte, no auto-increment
        +    if (Wire.endTransmission(false) != 0) return -1;
        +    if (Wire.requestFrom((uint8_t)ADDR_TLC, (uint8_t)1) != 1) return -1;
        +    return (int16_t)Wire.read();
        +}
        +
         static bool tlc_init() {
             const uint8_t mode[2] = {0x00, 0x00};       // MODE1: oscillator on, no sub/all-
call; MODE2: dimming, defaults
             if (!tlc_write(0x00, mode, 2)) return false;
             delayMicroseconds(600);                      // oscillator start-up (500 us max)
        +    bool ok = true;
             uint8_t zero[16] = {0};
        -    tlc_write(0x02, zero, 16);                   // PWM0..15 = 0
        +    ok &= tlc_write(0x02, zero, 16);             // PWM0..15 = 0
             const uint8_t grp[2] = {0xFF, 0x00};         // GRPPWM, GRPFREQ (group dimming
unused: LDR=10)
        -    tlc_write(0x12, grp, 2);
        +    ok &= tlc_write(0x12, grp, 2);
             uint8_t ledout[4] = {0, 0, 0, 0};
             for (uint8_t o = 0; o < 16; o++)
                 if (OUT_GROUP[o] >= 0) ledout[o / 4] |= (uint8_t)(0x2 << (2 * (o % 4)));   //
LDRx = 10: individual PWM
        -    tlc_write(0x14, ledout, 4);
        +    ok &= tlc_write(0x14, ledout, 4);
             iref_code = 0xFF;
             const uint8_t iref = iref_code;              // CM=1 HC=1 CC=63 -> CG 0.992 (full
scale)
        -    tlc_write(0x1C, &iref, 1);
        +    ok &= tlc_write(0x1C, &iref, 1);
             memset(last_pwm, 0, sizeof(last_pwm));
        -    return true;
```

```
        +    return ok;                                      // any failed write -> caller
retries the whole init
         }

         // current-gain steps (IREF register {CM,HC,CC[5:0]}): CG = (1+HC)(1+CC/64)/4 *
3^(CM-1)
         static const struct { uint8_t code; float cg; } CG_STEPS[] = {
             {0x00, 0.0833f}, {0x3F, 0.1654f}, {0x7F, 0.3307f}, {0xBF, 0.4961f}, {0xFF,
0.9922f}};

        -static void tlc_output(const float drive[G_COUNT]) {
        +static bool tlc_output(const float drive[G_COUNT]) {   // false on a bus error:
caller re-initialises
             float mx = 0;
             for (uint8_t g = 0; g < G_COUNT; g++) if (drive[g] > mx) mx = drive[g];
             uint8_t k = 0;
        @@ -65,12 +75,29 @@
                 pwm[o] = (uint8_t)(p > 255 ? 255 : (p < 0 ? 0 : p));
                 changed |= pwm[o] != last_pwm[o];
             }
        -    if (CG_STEPS[k].code != iref_code) {
        -        iref_code = CG_STEPS[k].code;
        -        tlc_write(0x1C, &iref_code, 1);
        -        changed = true;
        +    const uint8_t code = CG_STEPS[k].code;
        +    if (code != iref_code) {
        +        // IREF acts when written; PWM acts at STOP (MODE2.OCH=0 covers 02h..17h
only). Order the writes so the
        +        // in-between state is never brighter than either end: gain up -> PWM first,
gain down -> IREF first.
        +        const bool gain_up = code > iref_code;   // codes in CG_STEPS increase with
gain
        +        if (gain_up) {
        +            if (!tlc_write(0x02, pwm, 16)) return false;
        +            memcpy(last_pwm, pwm, 16);
        +            if (!tlc_write(0x1C, &code, 1)) return false;
        +        } else {
        +            if (!tlc_write(0x1C, &code, 1)) return false;
        +            iref_code = code;
        +            if (!tlc_write(0x02, pwm, 16)) return false;
        +            memcpy(last_pwm, pwm, 16);
        +        }
        +        iref_code = code;
        +        return true;
        +    }
        +    if (changed) {
        +        if (!tlc_write(0x02, pwm, 16)) return false;
        +        memcpy(last_pwm, pwm, 16);
             }
        -    if (changed && tlc_write(0x02, pwm, 16)) memcpy(last_pwm, pwm, 16);
        +    return true;
         }

         static void tlc_sleep() {
        @@ -81,14 +108,23 @@
         }

         // ---------------------------------------------------------------------- LM75B
        +static bool lm75_config(uint8_t conf) {        // Conf register (pointer 01h): B0 =
SHUTDOWN
```

```
        +    Wire.beginTransmission(ADDR_LM75);
        +    Wire.write((uint8_t)0x01);
        +    Wire.write(conf);
        +    return Wire.endTransmission() == 0;
        +}
        +
         static bool lm75_read(float *t) {
             Wire.beginTransmission(ADDR_LM75);
             Wire.write((uint8_t)0x00);
             if (Wire.endTransmission(false) != 0) return false;
             if (Wire.requestFrom((uint8_t)ADDR_LM75, (uint8_t)2) != 2) return false;
        -    int16_t raw = (int16_t)((Wire.read() << 8) | Wire.read());
        -    *t = (float)(raw >> 5) * 0.125f;              // 11-bit, 0.125 C/LSB
        -    return true;
        +    const uint8_t msb = (uint8_t)Wire.read();     // two statements: operand
evaluation order is unspecified
        +    const uint8_t lsb = (uint8_t)Wire.read();
        +    const int16_t raw = (int16_t)(((uint16_t)msb << 8) | lsb);
        +    *t = (float)(raw >> 5) * 0.125f;              // 11-bit two's complement, 0.125
C/LSB (arithmetic shift)
        +    return *t > -55.0f && *t < 125.0f;            // outside the sensor's range =
treat as a failed read
         }

         // ------------------------------------------------ supply (= battery) voltage
        @@ -100,12 +136,15 @@
             ADC0.SAMPCTRL = 16;
             ADC0.MUXPOS = ADC_MUXPOS_INTREF_gc;
             ADC0.CTRLA = ADC_ENABLE_bm;                   // 10-bit
        +    VREF.CTRLB |= VREF_ADC0REFEN_bm;              // keep INTREF running for the
whole burst ...
        +    delayMicroseconds(50);                        // ... and let it start (TStart 25
us typ, Table 36-22)
             uint32_t acc = 0;
        -    for (uint8_t i = 0; i < 8; i++) {
        +    for (uint8_t i = 0; i < 9; i++) {
                 ADC0.COMMAND = ADC_STCONV_bm;
                 while (ADC0.COMMAND & ADC_STCONV_bm) {}
        -        acc += ADC0.RES;
        +        if (i) acc += ADC0.RES;                   // discard the first conversion
after the reference starts
             }
        +    VREF.CTRLB &= (uint8_t)~VREF_ADC0REFEN_bm;
             uint16_t res = (uint16_t)(acc / 8);
             return res ? (uint16_t)((1100UL * 1023UL) / res) : 0;
         }
        @@ -115,27 +154,40 @@

         static void sleep_until_resumed() {
             tlc_sleep();
        -    Wire.beginTransmission(ADDR_LM75);           // LM75B shutdown (config register
bit 0)
        -    Wire.write((uint8_t)0x01);
        -    Wire.write((uint8_t)0x01);
        -    Wire.endTransmission();
        +    lm75_config(0x01);                           // LM75B shutdown; setup() wakes it
again after the reset below
             while (RTC.STATUS) {}
             RTC.CLKSEL = RTC_CLKSEL_INT32K_gc;
             while (RTC.PITSTATUS) {}
```

```
             RTC.PITINTCTRL = RTC_PI_bm;
             RTC.PITCTRLA = RTC_PERIOD_CYC32768_gc | RTC_PITEN_bm;   // wake every ~1 s
             set_sleep_mode(SLEEP_MODE_PWR_DOWN);
         +    uint8_t good = 0;
             for (;;) {
                 sleep_enable();
                 sleep_cpu();
                 sleep_disable();
         -       if (vdd_mv() >= VBAT_RESUME_MV) break;    // e.g. USB plugged in
         +       __builtin_avr_wdr();                      // WDT (2 s) keeps running in
power-down; PIT wakes every 1 s
         +       good = (vdd_mv() >= VBAT_RESUME_MV) ? (uint8_t)(good + 1) : 0;
         +       if (good >= 3) break;                     // 3 consecutive 1 s readings (e.g.
USB plugged in)
             }
             RTC.PITCTRLA = 0;
             _PROTECTED_WRITE(RSTCTRL.SWRR, RSTCTRL_SWRE_bm);   // clean restart
          }

          // ---------------------------------------------------------------------- main
         +// A reset in the middle of a transfer can leave a slave holding SDA low (the
TLC59116 has no bus time-out):
         +// clock SCL until SDA is released, then send a STOP. Pins are open-drain here (PORT
OUT = 0, external pull-ups).
         +static void i2c_bus_recover() {
         +    pinMode(PIN_PB0, INPUT);
         +    pinMode(PIN_PB1, INPUT);
         +    for (uint8_t i = 0; i < 9 && digitalReadFast(PIN_PB1) == LOW; i++) {
         +        pinMode(PIN_PB0, OUTPUT); delayMicroseconds(5);   // SCL low
         +        pinMode(PIN_PB0, INPUT);  delayMicroseconds(5);   // SCL released
         +    }
         +    pinMode(PIN_PB1, OUTPUT); delayMicroseconds(5);       // SDA low while SCL high
...
         +    pinMode(PIN_PB1, INPUT);  delayMicroseconds(5);       // ... then high = STOP
         +}
         +
          static void unused_pins_low_power() {
             const uint8_t pins[] = {PIN_PA2, PIN_PA3, PIN_PA7, PIN_PB2, PIN_PB3, PIN_PB4,
PIN_PB5,
                                     PIN_PC0, PIN_PC1, PIN_PC2, PIN_PC3};
         @@ -153,9 +205,12 @@
          #else
             pinMode(PIN_PA1, INPUT_PULLUP);
          #endif
         +    i2c_bus_recover();
             Wire.begin();
             Wire.setClock(I2C_HZ);
             delay(5);                                         // LED board rails settle
         +    lm75_config(0x00);                               // normal mode: undo a shutdown
left by sleep_until_resumed()
         +    _PROTECTED_WRITE(WDT.CTRLA, WDT_PERIOD_2KCLK_gc);   // 2 s watchdog: a hung loop
resets, LEDs re-init
             saved_t saved;
             EEPROM.get(0, saved);
             core_init(&core, &saved, millis());
         @@ -163,7 +218,7 @@
          }

          void loop() {
         -    static uint32_t last_ms, last_t, last_v, last_tlc_retry;
```

```
        +    static uint32_t last_ms, last_t, last_v, last_tlc_retry, last_tlc_check;
             uint32_t now = millis();
             if (now != last_ms) {
                 last_ms = now;
        @@ -172,7 +227,7 @@
                 core_buttons(&core, pressed, now);
                 if (now - last_t >= 1000) {
                     last_t = now;
        -            float t;
        +            float t = 0.0f;
                     bool ok = lm75_read(&t);
                     core_temperature(&core, ok, t, now);
         #ifdef DEBUG_SERIAL
        @@ -190,7 +245,12 @@
                     last_tlc_retry = now;
                     tlc_ok = tlc_init();
                 }
        -        if (tlc_ok && (now & 3) == 0) tlc_output(core.drive);   // 250 Hz update,
only writes on change
        +        if (tlc_ok && (now & 3) == 0 && !tlc_output(core.drive)) tlc_ok = false;   //
250 Hz, writes on change
        +        if (tlc_ok && now - last_tlc_check >= 1000) {   // LED board POR (cable, VLED
dropout) resets MODE1 to 11h
        +            last_tlc_check = now;
        +            int16_t m1 = tlc_read(0x00);
        +            if (m1 < 0 || (m1 & 0x11)) tlc_ok = false;  // OSC or ALLCALL set = not
our configuration -> re-init
        +        }
                 if (core.save_request) {
                     core.save_request = false;
                     saved_t s;
        @@ -200,6 +260,7 @@
                 if (core.batt == 2 && core.drive[G_WW] + core.drive[G_CW] + core.drive[G_R] +
core.drive[G_G] + core.drive[G_B] < 1e-4f)
                     sleep_until_resumed();
             }
        +    __builtin_avr_wdr();
             set_sleep_mode(SLEEP_MODE_IDLE);
             sleep_mode();                                    // wakes on the millis timer
interrupt
         }
        --- a/firmware/platformio.ini
        +++ b/firmware/platformio.ini
        @@ -15,6 +15,7 @@
         upload_protocol = serialupdi
         upload_speed = 230400
         build_flags = -Wall -Wextra -Iinclude
        +build_unflags = -w
         lib_deps =
             core

        @@ -24,7 +25,7 @@

         [env:native]
         platform = native
        -build_flags = -std=gnu++17 -Iinclude -lm
        +build_flags = -std=gnu++17 -Wall -Wextra -Iinclude -lm
         test_build_src = no
         lib_deps =
             core
```

```
        --- a/firmware/tools/gen_modes.py
        +++ b/firmware/tools/gen_modes.py
        @@ -3,7 +3,7 @@
         Groups: WW (4 x 2700K 2835), CW (4 x 5000K 2835), R, G, B (two 5050 RGB LEDs, each
die on its own channel). For each white mode we solve the
         luminance mix that lands exactly on the Planckian locus at the target CCT (CIE 1931
xy), using WW + CW and a
         little G (or B) to correct Duv. Night modes follow docs/research/light-modes.md: red,
or red + a little 2700K,
        -never green. Drive ratios are normalised so the strongest group = 1.0 (= full
TLC59116 current, 30 mA).
        +never green. Drive ratios are normalised so the strongest group = 1.0 (= full
TLC59116 current, 25 mA).

         LED data below is nominal datasheet data (VERIFY against the received bins; see
docs/design/electronics.md).
         Usage: python3 firmware/tools/gen_modes.py  [writes include/modes_table.h and prints
a summary]
        @@ -16,12 +16,12 @@
         import numpy as np

         # name: (x, y, lumens per LED at 30 mA, LEDs in group)
        -LED = {
        -    "WW": (0.4599, 0.4106, 12.0, 4),   # Bridgelux BXEN-27S Thrive 2700K CRI97 (ANSI
centre), ~12 lm @30 mA (scaled from 51 lm @150 mA)
        -    "CW": (0.3451, 0.3516, 13.5, 4),   # BXEN-50S 5000K CRI97
        -    "R": (0.700, 0.299, 4.4, 2),       # 2 x Kingbright KAAF-5050RGBS 625 nm, 1.4 cd
-> ~4.4 lm each (Lambertian, pi*Iv)
        -    "G": (0.170, 0.700, 4.4, 2),       # 525 nm, 1.4 cd
        -    "B": (0.131, 0.075, 1.3, 2),       # 470 nm, 0.42 cd
        +LED = {   # lumens per LED at 25.0 mA (TLC59116, REXT 750 R). Datasheet typicals:
VERIFY against received parts.
        +    "WW": (0.4599, 0.4106, 8.5, 4),    # Bridgelux BXEN-27S 2700K CRI97: 51 lm @150
mA x ~0.20 @30 mA (DS312 Fig. 10) x 25/30
        +    "CW": (0.3451, 0.3516, 9.3, 4),    # BXEN-50S 5000K CRI97: 56 lm @150 mA, same
scaling
        +    "R": (0.700, 0.299, 1.06, 2),      # 2 x Wurth 150141M173100 red 625 nm: 270 mcd
typ @20 mA -> pi*Iv x 25/20
        +    "G": (0.170, 0.700, 3.73, 2),      # green 520 nm: 950 mcd typ @20 mA (xy VERIFY:
520 nm sits greener than this)
        +    "B": (0.131, 0.075, 0.90, 2),      # blue 470 nm: 230 mcd typ @20 mA
         }
         GROUPS = ["WW", "CW", "R", "G", "B"]

        --- a/firmware/include/modes_table.h
        +++ b/firmware/include/modes_table.h
        @@ -9,14 +9,14 @@
         #define MODE_COUNT 10

         static const mode_t_ MODES[MODE_COUNT] = {
        -    {"Warm white 2700K", {4096, 0, 0, 0, 0}, 9, 15},  // WARM_2700: WW only; xy=
(0.4599,0.4106), 48 lm at full drive
        -    {"Soft white 3000K", {4096, 647, 0, 565, 0}, 10, 15},  // WHITE_3000: WW+CW+G,
Duv 0 by construction; xy=(0.4369,0.4041), 58 lm at full drive
        -    {"Neutral reading 4000K", {2749, 4096, 0, 1228, 0}, 12, 15},  // WHITE_4000:
WW+CW+G, Duv 0 by construction; xy=(0.3805,0.3768), 89 lm at full drive
        -    {"Daylight 5000K", {0, 4096, 0, 0, 0}, 12, 15},  // WHITE_5000: WW+CW+G, Duv 0 by
construction; xy=(0.3451,0.3516), 54 lm at full drive
        -    {"Blue-white 6500K", {0, 4096, 0, 1313, 2756}, 12, 15},  // COOL_6500: CW + B +
G; xy=(0.3136,0.3237), 59 lm at full drive
```

```
        -    {"Candlelight 1900K", {254, 0, 4096, 3682, 0}, 7, 13},  // CANDLE_1900: WW + R +
a little G; xy=(0.5389,0.4103), 20 lm at full drive
        -    {"Soft orange night light", {322, 0, 4096, 0, 0}, 4, 10},  // NIGHT_ORANGE: R 70%
+ WW 30% by luminance; no green; xy=(0.6429,0.3255), 13 lm at full drive
        -    {"Red night light", {0, 0, 4096, 0, 0}, 3, 11},  // NIGHT_RED: R only; xy=
(0.7000,0.2990), 9 lm at full drive
        -    {"Green", {0, 0, 0, 4096, 0}, 8, 15},  // GREEN: G only; xy=(0.1700,0.7000), 9 lm
at full drive
        -    {"Blue", {0, 0, 0, 0, 4096}, 8, 15},  // BLUE: B only; xy=(0.1310,0.0750), 3 lm
at full drive
        +    {"Warm white 2700K", {4096, 0, 0, 0, 0}, 9, 15},  // WARM_2700: WW only; xy=
(0.4599,0.4106), 34 lm at full drive
        +    {"Soft white 3000K", {4096, 665, 0, 472, 0}, 10, 15},  // WHITE_3000: WW+CW+G,
Duv 0 by construction; xy=(0.4369,0.4041), 41 lm at full drive
        +    {"Neutral reading 4000K", {2674, 4096, 0, 998, 0}, 12, 15},  // WHITE_4000:
WW+CW+G, Duv 0 by construction; xy=(0.3805,0.3768), 61 lm at full drive
        +    {"Daylight 5000K", {0, 4096, 0, 0, 0}, 12, 15},  // WHITE_5000: WW+CW+G, Duv 0 by
construction; xy=(0.3451,0.3516), 37 lm at full drive
        +    {"Blue-white 6500K", {0, 4096, 0, 1067, 2742}, 12, 15},  // COOL_6500: CW + B +
G; xy=(0.3136,0.3237), 40 lm at full drive
        +    {"Candlelight 1900K", {86, 0, 4096, 1046, 0}, 7, 13},  // CANDLE_1900: WW + R + a
little G; xy=(0.5389,0.4103), 5 lm at full drive
        +    {"Soft orange night light", {109, 0, 4096, 0, 0}, 4, 10},  // NIGHT_ORANGE: R 70%
+ WW 30% by luminance; no green; xy=(0.6429,0.3255), 3 lm at full drive
        +    {"Red night light", {0, 0, 4096, 0, 0}, 3, 11},  // NIGHT_RED: R only; xy=
(0.7000,0.2990), 2 lm at full drive
        +    {"Green", {0, 0, 0, 4096, 0}, 8, 15},  // GREEN: G only; xy=(0.1700,0.7000), 7 lm
at full drive
        +    {"Blue", {0, 0, 0, 0, 4096}, 8, 15},  // BLUE: B only; xy=(0.1310,0.0750), 2 lm
at full drive
         };
```